

React Interview — 10-Year-Level Answers

The questions and company labels below follow the React interview material previously discussed. Answers are intentionally written to sound practical, senior and production-oriented.

1. What's the difference between a Single Page Application and Multi Page Application?

Company / source: General React interview

10-year answer: In an SPA, the browser generally loads the application shell once and subsequent navigation is handled on the client without a complete page reload. In an MPA, navigation typically results in the browser requesting a new HTML page from the server. In enterprise applications, I don't choose SPA just because React supports it. I look at SEO requirements, application size, navigation patterns, initial-load performance and whether server rendering is beneficial. Internal dashboards and complex authenticated applications are often good SPA candidates.

■ **Standout:** The architecture decision should come from application requirements, not simply from the frontend framework.

2. What are the advantages and disadvantages of React?

Company / source: General

10-year answer: React gives us component-based architecture, reusable UI, a strong ecosystem and predictable one-way data flow. Its flexibility is also a challenge at enterprise scale because teams can adopt inconsistent patterns for state, API handling and component structure. I therefore consider coding standards, component boundaries, state ownership, testing and performance guidelines important parts of a React architecture.

■ **Standout:** React gives flexibility, but at enterprise scale that flexibility needs architectural discipline.

3. Is React one-way or two-way data flow?

Company / source: General

10-year answer: React follows one-way data flow. Data generally flows from parent to child through props. A child communicates back through a callback supplied by the parent. A form may look like two-way binding from the user's perspective, but technically the state ownership and data flow remain explicit.

■ **Standout:** The interaction can feel bidirectional, but state ownership and data flow remain explicit.

4. Real DOM vs Virtual DOM?

Company / source: General

10-year answer: The real DOM is the browser's actual document structure. React maintains an in-memory representation and uses reconciliation to determine what changed between renders, then commits the required changes to the real DOM. I would not simply say Virtual DOM is 'faster than DOM'; the larger benefit is React's declarative model and efficient reconciliation.

■ **Standout:** Virtual DOM isn't magic performance; reconciliation and minimizing unnecessary DOM work are the important parts.

5. Functional component vs class component?

Company / source: General

10-year answer: Functional components are JavaScript functions that return UI and can use Hooks for state and other React features. Class components use lifecycle methods and class state. For new development I prefer functional components because Hooks make logic reuse and composition easier. For legacy enterprise applications, however, I would modernize incrementally where that reduces risk rather than automatically rewriting everything.

■ **Standout:** For legacy systems, modernization doesn't necessarily mean rewriting; I look at risk and business value.

6. Is a functional component stateless or stateful?

Company / source: General

10-year answer: A functional component can be either. Before Hooks, functional components were commonly used as stateless components, but today a functional component can maintain state, run effects and manage complex behavior.

■ **Standout:** The distinction is no longer function = stateless and class = stateful.

7. Which Hooks have you used besides useState and useEffect?

Company / source: BMW / Coforge

10-year answer: I've used useRef, useMemo, useCallback and useReducer, and I've worked with custom Hooks. I use them based on the problem rather than using every Hook by default. useRef is useful for persistence without a render, useMemo for expensive derived calculations, useCallback for useful referential stability, and useReducer when state transitions become structured.

■ **Standout:** Senior signal: I choose a Hook based on the rendering and state problem, not because the Hook exists.

8. Why can't Hooks be called inside loops or conditions?

Company / source: Coforge

10-year answer: React depends on Hooks being called in the same order on every render. If a Hook is conditionally executed, the order can change between renders and React can no longer reliably associate Hook state with the correct call. That is why Hooks must be called at the top level of a component or custom Hook.

■ **Standout:** The rule isn't arbitrary; it exists because React relies on consistent Hook call order.

9. What are the side effects of useEffect?

Company / source: General

10-year answer: useEffect is used to synchronize React with something outside the rendering calculation—for example API requests, subscriptions, timers, browser event listeners or external services. I don't use useEffect simply to calculate derived values if they can be calculated directly from props and state. I also make sure effects clean up subscriptions, timers and listeners.

■ **Standout:** I treat useEffect as synchronization with an external system, not as a general-purpose place for business logic.

10. useEffect vs useLayoutEffect?

Company / source: BMW

10-year answer: useEffect is for normal post-render side effects. useLayoutEffect runs synchronously after DOM mutations but before the browser paints, so it is useful for DOM measurement or a visual adjustment that must happen before the user sees it. I don't use it as a default replacement for useEffect because it can block painting.

■ **Standout:** useLayoutEffect is about timing-sensitive DOM work; it shouldn't be the default choice.

11. Have you created custom Hooks? What kind?

Company / source: General

10-year answer: I use custom Hooks when behavior needs to be reused across components—for example API-related state handling, debounced values, event listeners or reusable UI behavior. I keep each Hook focused rather than building one generic Hook that tries to solve every problem.

■ **Standout:** I extract a custom Hook when I'm reusing behavior, not simply because a component has become long.

12. What's the difference between state and props?

Company / source: General

10-year answer: Props are inputs provided by a parent and should be treated as read-only by the child. State is owned by the component or its state-management layer and changes over time. The architectural question I focus on is ownership: where should the source of truth live? I keep state close to the components that need it and lift it when multiple components need coordinated access.

■ **Standout:** State ownership is more important than simply deciding whether something is a prop or state.

13. Are React states mutable or immutable?

Company / source: General

10-year answer: React state should be treated as immutable. Instead of directly modifying an existing object or array, I create a new value and update state with it. This gives predictable transitions and makes reference-based change detection reliable.

■ **Standout:** Immutability gives predictable state transitions and makes change detection easier.

14. Controlled vs uncontrolled components?

Company / source: General

10-year answer: In a controlled component, React state is the source of truth for the input. In an uncontrolled component, the DOM maintains the value and a ref can be used to access it. I generally prefer controlled inputs for business forms because validation, conditional UI and submission logic are easier to coordinate. Uncontrolled inputs can still be appropriate for simpler scenarios.

■ **Standout:** Controlled components give stronger control over business rules; uncontrolled components reduce state synchronization when that control isn't required.

15. What is an HOC?

Company / source: General

10-year answer: A Higher-Order Component is a function that takes a component and returns an enhanced component. Historically it was common for authentication, permissions and subscriptions. In modern React, many HOC use cases can be handled more cleanly with Hooks and composition, but I still understand HOCs because they appear in legacy enterprise code and libraries.

■ **Standout:** I wouldn't introduce an HOC into new code just because it is possible; I first check whether composition or a custom Hook is cleaner.

16. Have you used Redux? Explain the complete flow.

Company / source: BMW / Coforge

10-year answer: The typical flow is UI → dispatch action → middleware if required → reducer → store update → subscribed components. For example, an e-commerce Add to Cart action is dispatched, middleware can handle asynchronous work, the reducer calculates the next state, and components subscribed to the relevant slice update. I also pay attention to selectors and state shape because Redux does not automatically make an application performant.

■ **Standout:** Redux gives predictable state transitions, but good state shape and selective subscriptions make it scalable.

17. Redux vs Context API?

Company / source: BMW / Coforge

10-year answer: Context provides a mechanism for sharing values through the component tree without prop drilling. Redux provides a more structured state-management architecture around actions, reducers, middleware and subscriptions. I might use Context for relatively stable concerns such as theme or locale and Redux for complex application state with structured transitions and broader debugging needs. I would not decide solely on application size.

■ **Standout:** Context is a communication mechanism; Redux is a state-management architecture.

18. Why not store large datasets in Context?

Company / source: BMW / Coforge

10-year answer: The size alone is not the main issue. The important factors are update frequency and number of consumers. If a broad Context value changes frequently, many consumers may be affected even if they only care about a small part of the value. I would consider splitting contexts, moving state closer to consumers, or using selective subscriptions.

■ **Standout:** The Context problem is primarily update granularity, not simply the number of records.

19. Why does Context cause re-renders?

Company / source: BMW / Coforge

10-year answer: When a provider value changes, React needs to notify consumers. A common issue is creating a new object as the provider value on every render, such as `value={{user, updateUser}}`. Even if the underlying data is unchanged, the object reference can be new. I would profile first and then consider Context boundaries, stable values or different state ownership.

■ **Standout:** Understanding reference identity is critical when diagnosing Context-related renders.

20. Why create multiple Contexts?

Company / source: BMW / Coforge

10-year answer: Multiple Contexts can improve update isolation. Instead of one huge application context containing user, theme, notifications and frequently changing dashboard data, I can separate meaningful domains. I would not create a Context for every variable; boundaries should represent meaningful ownership and update patterns.

■ **Standout:** I split Contexts around ownership and update frequency, not arbitrarily around individual fields.

21. How can you prevent unnecessary re-rendering?

Company / source: BMW / Cognizant / Coforge

10-year answer: I don't start by adding `React.memo` everywhere. First I identify the cause using React DevTools Profiler. Then I check state ownership, parent renders, unstable object/function references, broad Context updates, expensive calculations and large lists. Depending on the bottleneck, I might use memoization, better state placement, selectors, Context splitting or virtualization.

■ **Standout:** I optimize the cause of the render, not the render itself.

22. What is React.memo?

Company / source: BMW / Cognizant

10-year answer: `React.memo` memoizes a functional component and can skip rendering when its props have not changed according to its comparison. It is most useful when the component is expensive enough to matter and receives stable props. If I pass a newly created object or function every time, memoization may not help. I also consider the comparison and memory overhead.

■ **Standout:** Memoization is an optimization tool, not a default wrapper for every component.

23. Parent counter changes. Child is React.memo with no props. Will it render?

Company / source: BMW / Cognizant

10-year answer: If the child is memoized and receives no props, a parent state update does not by itself require the child to render again. React can reuse the memoized result.

■ **Standout:** Parent render does not automatically mean a memoized child must render.

24. What if the child receives a primitive prop?

Company / source: BMW / Cognizant

10-year answer: If the child receives a primitive such as a string and the value remains the same, `React.memo`'s shallow comparison sees the prop as unchanged and can skip the child render.

■ **Standout:** Primitive equality makes memoization straightforward when the value is unchanged.

25. What if the child receives an inline object?

Company / source: BMW / Cognizant

10-year answer: An inline object is newly created on every parent render. Even if its contents are identical, its reference is different. `React.memo` uses shallow comparison by default, so it sees that prop as changed and the child can render again. This is why JavaScript reference equality matters in React performance work.

■ **Standout:** React performance requires understanding JavaScript reference identity, not just React APIs.

26. What is React Fiber?

Company / source: BMW

10-year answer: Fiber is React's internal reconciliation architecture for representing rendering work and managing scheduling. It allows React to break rendering work into units and manage when work should be performed. Application developers normally don't manipulate Fiber directly, but understanding it helps explain modern React's rendering and prioritization behavior.

■ **Standout:** Fiber is an implementation architecture behind reconciliation and scheduling; it isn't something I directly program against.

27. What is React Profiler?

Company / source: BMW / Coforge

10-year answer: React Profiler helps identify which components rendered, how long rendering took and which interaction triggered the work. If a dashboard becomes slow when changing a filter, I would profile that interaction instead of immediately adding memoization. The profiler helps determine whether the issue is a large list, expensive calculation, Context update or component render.

■ **Standout:** Profiler tells me where time is going; it doesn't tell me that memoization is automatically the solution.

28. How would you handle 20,000 or 50,000 rows?

Company / source: Cognizant

10-year answer: I would first question whether all records need to be transferred and rendered by the browser. Where the requirement allows it, I'd use server-side pagination, filtering and sorting. If users need a large scrolling dataset, I'd use virtualization so only visible rows are mounted. I'd also optimize row components, use stable keys and avoid expensive work per row.

■ **Standout:** Virtualization solves the rendering problem; it doesn't solve the network or data-transfer problem.

29. What is virtualization?

Company / source: Cognizant

10-year answer: Virtualization means rendering only the items currently visible in the viewport instead of mounting every item in the dataset. If a table has 50,000 rows but the viewport displays only 30, only a small set needs to be mounted at a time. I would often combine this with server-side data strategies.

■ **Standout:** Virtualization reduces DOM/rendering work; it does not reduce the size of the source dataset by itself.

30. How would you debug a lagging React list?

Company / source: Coforge

10-year answer: I would reproduce the exact interaction and profile it. Then I would separate React rendering cost, JavaScript computation, DOM volume and network latency because each needs a different solution. I would inspect list keys, unnecessary renders, expensive transformations and whether virtualization or server-side processing is appropriate.

■ **Standout:** I separate data latency, JavaScript computation and DOM rendering because they require different solutions.

31. What is useMemo?

Company / source: BMW / Cognizant

10-year answer: useMemo memoizes the result of a calculation until dependencies change. I use it when the calculation is genuinely expensive or when a stable derived value has a downstream purpose. I don't add it simply because a component renders; cheap calculations may not justify the extra complexity and dependency management.

■ **Standout:** I use useMemo for a measured calculation or referential-stability need, not as decoration.

32. useMemo vs useCallback?

Company / source: BMW / Cognizant

10-year answer: useMemo memoizes a value; useCallback memoizes a function reference. useCallback is useful when referential stability matters, such as passing a callback to a memoized child or using it in dependency-sensitive logic. Both should be introduced when they solve a real problem.

■ **Standout:** useMemo → value/result. useCallback → function reference.

33. How do you compare old and new prop values?

Company / source: Coforge

10-year answer: I can use useRef to retain the previous value across renders, compare it with the current prop, and then update the ref for the next render. I prefer a ref because I need persistence across renders but don't need the previous value itself to trigger another render.

■ **Standout:** I use useRef when I need history without making that history part of the rendered state.

34. Why useRef instead of useState?

Company / source: Coforge

10-year answer: Updating state schedules a render. Updating ref.current does not. If I only need to remember a previous value, DOM node, timer ID or mutable instance value without updating the UI, useRef is a better fit.

■ **Standout:** State = remember and update UI. Ref = remember without updating UI.

35. How do you close a modal when clicking outside but not inside?

Company / source: Coforge

10-year answer: I'd keep a ref to the modal element and handle the click at an appropriate level. On a click, I check whether the event target is contained inside the modal using contains(). If it isn't, I close the modal. I wouldn't use modal width/height or coordinate calculations because responsive layouts make that fragile.

■ **Standout:** I use DOM containment rather than coordinates or dimensions.

36. What if multiple APIs are called every time the component renders?

Company / source: Coforge

10-year answer: First I'd determine why the API call is tied to rendering. Requests should be controlled through an effect or a data-fetching abstraction with well-defined dependencies. I'd check for unstable dependencies and duplicate fetching. If requests are independent, I'd also run them concurrently.

■ **Standout:** An API request should be tied to a deliberate data lifecycle, not accidentally to every render.

37. How would you fetch two APIs simultaneously?

Company / source: Coforge

10-year answer: If the APIs are independent, I would initiate them concurrently rather than waiting for the first to finish. With Promises, Promise.all is straightforward; with RxJS, forkJoin can be appropriate when I need the final result from multiple observables.

■ **Standout:** The goal is to remove unnecessary sequential latency when dependencies are independent.

38. How would you refresh API data every 30 seconds?

Company / source: Coforge

10-year answer: I'd use a timer/polling mechanism with proper cleanup, or a data-fetching library that supports polling. I would also consider whether polling is really appropriate or whether the requirement is better served by WebSockets or server-sent events. The important part is avoiding timers that survive unmounting.

■ **Standout:** Polling is a solution, but I first check whether the requirement really needs polling or an event-driven approach.

39. How do you handle an API that takes a long time to respond?

Company / source: Coforge

10-year answer: I represent loading explicitly and, depending on the UI, use skeletons, progress or a meaningful loading state. I handle timeout and error states, avoid duplicate requests, and consider cancellation when a request becomes stale. I would also investigate the backend latency rather than hiding it behind an infinite spinner.

■ **Standout:** I separate user-experience handling from solving the underlying API latency.

40. How do you implement protected routes?

Company / source: Coforge

10-year answer: I'd maintain authentication state and enforce route access at the routing layer, redirecting unauthenticated users appropriately. But frontend route protection is not a security boundary. Backend APIs must independently validate authentication and authorization.

■ **Standout:** A protected React route is not a security boundary by itself; authorization must be enforced on the server.

41. What is the difference between Webpack and Vite?

Company / source: Coforge

10-year answer: Webpack is a mature bundler that builds a dependency graph and bundles modules. Vite uses native ES modules during development with dependency pre-bundling where appropriate, reducing development startup and feedback work. Production still goes through an optimized build process.

■ **Standout:** Vite changes the development workflow; it doesn't remove the need for a production build.

42. Why is Vite faster than Webpack?

Company / source: Coforge

10-year answer: Traditional bundlers can spend significant time building a dependency graph before serving the application. Vite can leverage native browser ES modules and serve source files on demand, which reduces development startup work and can make HMR faster.

■ **Standout:** Vite's development speed largely comes from serving source modules on demand.

43. What is Module Federation?

Company / source: Coforge

10-year answer: Module Federation allows separately built applications to expose and consume modules at runtime. In a microfrontend architecture, a shell can consume independently owned remotes. In production I would think about shared dependency versions, compatibility, authentication, routing, failure isolation and independent delivery.

■ **Standout:** The difficult part of microfrontends is operational ownership and dependency management, not simply loading a remote.

44. What are Shell and Remote modules?

Company / source: Coforge

10-year answer: The Shell is the host application responsible for composition, navigation and common concerns. Remote applications own independently developed features consumed by the shell. Good boundaries should correspond to meaningful business or team ownership; otherwise microfrontends can add more complexity than value.

■ **Standout:** Microfrontend boundaries should support ownership and independent delivery.

45. Instead of React and Node, couldn't you use Next.js?

Company / source: Coforge

10-year answer: Next.js is a React framework that adds application-level capabilities such as routing and different rendering/data-fetching strategies. If an application benefits from SSR, SEO or framework-level conventions, Next.js can be a strong choice. For an internal SPA without those requirements, a standard React architecture may be simpler.

■ **Standout:** I choose the framework based on application requirements rather than treating Next.js as automatically superior.

46. Is your frontend server-side rendered or a static build?

Company / source: Coforge

10-year answer: In a traditional React SPA, the production process generates static assets that are served by a web server or CDN, while rendering primarily happens in the browser. SSR generates HTML on the server. I distinguish build output from rendering strategy: creating a production build does not automatically mean the application is server-side rendered.

■ **Standout:** A production build and SSR are different concepts.

47. What is hydration?

Company / source: React / Next.js

10-year answer: Hydration is where React attaches client-side behavior to HTML generated on the server. SSR gives the browser HTML earlier, but that HTML isn't automatically interactive. Hydration connects the React application to that server-rendered markup so event handlers and client behavior become active.

■ **Standout:** SSR generates HTML; hydration makes that server-rendered UI interactive.

48. Build a React counter.

Company / source: Multiple interviews

10-year answer: I'd use `useState` and a functional updater because the next value depends on the previous value: `setCount(prev => prev + 1)`. That approach remains correct when multiple updates are batched.

■ **Standout:** I use the functional updater whenever the next state depends on the previous state.

49. Create a `forEach` implementation.

Company / source: JavaScript / React interviews

10-year answer: `Array.prototype.myForEach = function(callback) { for (let i = 0; i < this.length; i++) { callback(this[i], i, this); } }`; The callback receives the current value, index and original array. Unlike `map`, `forEach` is intended for side effects and does not create a transformed array.

■ **Standout:** The important part is understanding the contract: value, index, array, and no transformed return array.

50. Doctor availability React task.

Company / source: Cognizant / Coforge

10-year answer: I'd keep the doctor list as the source of truth and store only the selected filter in state. The filtered list should be derived rather than stored separately. That avoids duplicated state becoming inconsistent.

■ **Standout:** I avoid storing derived state when it can be calculated reliably from the source of truth.